



ASSESSMENT & EVALUATION FOR REMOTE CANDIDATES - Aug 2026

Project Overview: Operations API, Payment gateway integration, RAG implementation.

Primary Objective: To consider your candidature for the Python Backend developer position.

This Take home Project will give our Team Member & other senior stakeholders an opportunity to assess & evaluate your technical skills and decide for potential next steps, if any.

Technical Objective: This project involves building the backend infrastructure for a Chatbot. Candidates will create a robust set of REST APIs that allow users to upload private documents to build a personal knowledge base, and query those documents using an AI assistant.

Technical Overview

1. Python - Django

You may use Django or another relevant framework, such as FastAPI, to implement this project.

2. LangChain (RAG Framework & Vector Workflows)

LangChain is an open-source orchestration framework designed to simplify the construction of applications powered by Large Language Models (LLMs).

Role in this Project: Instead of writing custom semantic search pipelines, you will leverage LangChain's built-in modules to handle the core mechanics of Retrieval-Augmented Generation (RAG). This includes using text splitters (e.g., RecursiveCharacterTextSplitter) to chunk uploaded files, creating vector store abstractions (e.g., Chroma, FAISS, or Pinecone) to index data, and orchestrating the final RAG chain to inject retrieved context directly into the LLM prompt.

3. OpenRouter (Unified LLM API & Free-Tier Access)

OpenRouter is an API aggregator that provides a single, unified interface to interact with dozens of different open-source and commercial AI models (including models from OpenAI, Anthropic, Meta, and Google).

Role in this Project: To ensure you can complete this take-home project completely free of charge, you will use OpenRouter as your LLM gateway. OpenRouter offers a robust selection of free-tier models (such as Mistral 7B, Gemma, or OpenChat variants). You will configure your



LangChain LLM class to route requests through OpenRouter's API endpoint, removing the need for paid personal API keys.

Project Start Date: 30 Aug

Last Project Submission Date: 31 Aug

Status: Take Home / Remote

Notes: Submitting early will improve your evaluation. Don't hesitate to reach out if you have any questions.

Project Requirements:

Part 1: Document Management & Vector Storage (LangChain)

Endpoints under this section allow users to build their private knowledge base. Uploaded files must be processed, chunked, embedded, and indexed inside a vector store via LangChain.

APIs to be Created

Document Upload API

- **Endpoint:** `/api/documents/upload/`
- **Method:** `POST`
- **Request Format:** Form-data with a file field.
- **Example Request:**
- Plaintext

Form-Data:

file: @knowledge_base.pdf

-
- **Response:**
 - **On Success (202 Accepted):** Task is accepted for asynchronous ingestion.
 - JSON

```
{  
  
  "message": "Document uploaded and ingestion started",  
  
  "document_id": "<document_id>",  
  
  "task_id": "<task_id>"  
}
```



}

-
- **On Failure (400 Bad Request):**
- JSON

{

"error": "Invalid file format. Only PDF, TXT, and Markdown files are allowed."

}

-

Document Ingestion Status API

- **Endpoint:** </api/documents/status/>
- **Method:** [GET](#)
- **Request Parameters:** [task_id=<task_id>](#)
- **Response:**
 - **Task Running:**
 - JSON

{

"task_id": "<task_id>",

"status": "PROCESSING"

}

-
- **Task Success:**
- JSON

{

"task_id": "<task_id>",

"status": "SUCCESS",

"message": "Document successfully parsed, embedded, and indexed in vector storage."

}

-
- **Task Failed:**
- JSON



```
{  
  "task_id": "<task_id>",  
  "status": "FAILURE",  
  "error": "Failed to parse document content."  
}  
  
○
```

Expected Background Pipeline Operations

- **Text Extraction:** Read raw content from the uploaded file type (PDF, TXT, or MD).
- **Chunking:** Use LangChain's text splitters (e.g., [RecursiveCharacterTextSplitter](#)) to split document content into readable, contextual chunks.
- **Vector Ingestion:** Calculate embeddings via an embedding model open-source or commercial, and store the vector chunks inside an isolated namespace linked to the uploading user's [user_id](#).

Part 2: RAG Chat Engine

This section connects the retrieved context to an LLM, validating that the user has an active subscription and sufficient daily token credits before rendering an answer.

APIs to be Created

RAG Query API

- **Endpoint:** [/api/chat/query/](#)
- **Method:** [POST](#)
- **Request Format:** JSON Body
- **Example Request:**
- JSON

```
{  
  "query": "What is the cancellation policy mentioned in the employee handbook?"  
}
```

-
- **Processing Rules:**
 - Use LangChain's Vector Store Retriever to search for context matches inside document collections.
 - Run the combined query and context prompt through the LLM.
- **Response:**



- On Success (200 OK):
- JSON

```
{  
  
  "answer": "The cancellation policy requires a written notice 14 days prior...",  
  
}
```

Part 3 (Bonus) Advanced Retrieval with HyDE (Hypothetical Document Embeddings)

Standard vector search compares the raw user query vector directly against indexed document vectors. Because questions and answers often differ significantly in length, syntax, and vocabulary, direct semantic matching can produce suboptimal similarity scores.

In this bonus section, candidates will implement **HyDE (Hypothetical Document Embeddings)** to bridge this semantic gap and elevate retrieval accuracy.

Gao et al., "Precise Zero-Shot Dense Retrieval without Relevance Labels" (ACL 2023) — arXiv:2212.10496 - <https://arxiv.org/pdf/2212.10496>

How HyDE Works in this Workflow

1. **Hypothetical Generation:** When a user submits a query, prompt an LLM (via OpenRouter) to draft a hypothetical, ideal passage that directly answers the question—even if hallucinated or factually approximate.
2. **Hypothetical Vector Embedding:** Embed this generated hypothetical document rather than the short raw query. The resulting vector captures domain-specific context and expected answer phrasing.
3. **Enhanced Vector Search:** Use the hypothetical document's embedding to query the vector store, retrieving the true source chunks with higher semantic alignment.
4. **Final RAG Synthesis:** Pass the retrieved real source context and the original question to the final LLM to generate the verified answer.

APIs to be Extended / Created

Advanced RAG Query API (HyDE Enabled)

- Endpoint: `/api/chat/query/`
- Method: `POST`
- Request Format: JSON Body

Example Request:

```
JSON  
{  
  "query": "What is the cancellation policy mentioned in the employee handbook?",
```



```
"use_hyde": true
}
```

Processing Rules:

- When **use_hyde: true**, route the query through your HyDE pipeline before running vector retrieval.
- If **use_hyde: false** (or omitted), fall back to standard baseline vector search.
- Log/return retrieval metadata showing both the generated hypothetical text and the final source chunks for debugging and grading visibility.

Example Response (200 OK):

```
JSON
{
  "answer": "The cancellation policy requires a written notice 14 days prior...",
  "retrieval_metadata": {
    "mode": "hyde",
    "hypothetical_passage": "The standard cancellation policy states that an employee or client must submit a formal written notice at least 14 days in advance to avoid penalties...",
    "retrieved_chunks_count": 3
  }
}
```

Evaluation Criteria for Bonus

- **LangChain Integration:** Clean composition of the HyDE pipeline using LangChain runnables/LCEL chains or custom retriever abstractions.
- **Latency & Fallback Handling:** Graceful error handling if the initial hypothetical generation fails or times out.
- **Toggle Flexibility:** Seamless switching between standard RAG and HyDE via the request payload.

Part 4 Dockerizing and Finalization

- Use Docker and Docker Compose to package your web application, database, Redis, Celery and dashboard services.

- When showcasing your application, include the necessary Docker commands to run the system in a README file.



American Tiger LLC
5900 Balcones Drive Suite 100
Austin, TX 78731

- Document your API using Postman, Bruno, OpenAPI, or another widely used API documentation tool.
 - Ensure the README also includes clear instructions on how to set up and run the entire application.
-

Please submit your Final Submission to:

Mr. Truong Vinh Phuoc “Matthew” at phuoc.truong@ravid.cloud

Closing Notes: Should you have any technical questions at any time, please contact:

Truong Vinh Phuoc “Matthew”, Back-End Developer & Full Stack Development Team.

CONFIDENTIAL